Telomy — Integration Instructions

How to take our app forward on your `healthagent` + Supabase backend · prepared for the Mervin / mclarenmervin team · October 2026

You're getting our full git repo (`anshumanastronaut/telomy`, branch `telomy/twin3d-sinc-design-ring`). This document is the step-by-step plan to merge it with your stack. The companion `docs/ARCHITECTURE_HANDOFF.md` in the repo has the deeper architecture; this is the "how to do it".

0. The plan in one line

Both backends are **FastAPI** and use the **same AI pattern** (retrieval → model → guardrail → compose). So: **converge on your Supabase as the data & auth bus**, and **fold our engine into `healthagent/app/analytics`**. Our mobile app talks to the same Supabase tables — exactly the "separate mobile team integrates via Supabase" contract in your `AGENTS.md`.

Layer	Today (ours)	Target (merged)
Data + auth + multi-user	SQLite per user + token auth	**Your Supabase** (Postgres + RLS + Auth)
Intelligence (engine + AI)	our FastAPI engine	merged into `healthagent/app/analytics` + `app/agent`
Devices	daily signals + high-freq vitals contract	same contract, writing to Supabase via your gateway/webhooks
Mobile app	React Native (Expo)	reads/writes the same Supabase tables

1. Prerequisites & access

- Reader (or dev) invite to the **Supabase project**; the project URL + **anon key** (for the app) and a **service-role key** or RLS policies (for the engine).
- The current **Supabase schema** (we can see `profiles`, `wellness_documents` (JSONB), `normalized_wellness`, `activity_sessions`).
- Clone our repo; backend is in `backend/` (Python 3.13, `pip install -r requirements.txt`, `pytest` — 168 tests). Mobile in `mobile/` (Expo).

2. Step-by-step

Step 1 — Expand the Supabase schema.

Our engine needs **queryable rows**, not one JSONB blob, so correlations, lab-history trends and high-frequency vitals work. Add these tables (all keyed by `user_id uuid references auth.users`, with RLS `auth.uid() = user_id`):

```
reports(id, user_id, title, panel, collected_on, source, is_test_data)
results(id, report_id, user_id, marker_id, value_num, value_text, unit, ref_low, ref_hig
signals(user_id, day, metric, value, source)                          -- daily wearable m
events(id, user_id, ts, kind, label, severity, data)                  -- logged life even
trackers(id, user_id, key, name, kind, plan, baseline, ...)           -- talk-to-track
tracker_entries(id, tracker_id, user_id, ts, value, note)
insights(id, user_id, kind, title, body, confidence, evidence, receipts, review_state)
vitals_ts(user_id, ts, metric, value, source, step_s)                 -- high-frequency t
imaging(id, user_id, data jsonb)                                      -- CT/MRI/DEXA find
science_findings(id, kind, payload, citation, evidence, active)       -- new science as d
```

The exact DDL mirrors `backend/app/db.py`. Keep your `wellness_documents` for the app's convenience if you like; the engine reads the relational tables.

Step 2 — Write the data-layer adapter.

Our engine reads/writes through one thin module, `backend/app/db.py` (`rows()`, `one()`, `exec_()`, `tx()`). Re-implement those four against Supabase using your `app/common/supabase_client.py` (or the Postgres connection). Because **every engine module goes through `db.*`**, swapping this one file points the whole engine at Supabase — no engine logic changes.

Step 3 — Merge the engine modules into `healthagent/app/analytics`.

Copy these from `backend/app/` into your analytics package (they're pure Python on `db.*` + numpy):

- `bioage.py` + `models/bioage_nhanes.json` — biological/fitness/muscle/heart/organ ages (NHANES-validated)
- `engine.py` — PhenoAge, effect/correlation engine (OLS + Benjamini-Hochberg), insights
- `predict.py` — PREVENT / PCE / diabetes / FIB-4 / KFRE risk
- `retrieval.py`, `cellular.py`, `science.py` — RAG, cellular KB, the science registry
- `trackers.py`, `nutrition.py`, `cgm.py`, `twin.py`, `whoop*.py`

Your `app/agent` (context_loader + graph + guardrails + llm + narration) already mirrors our AI pattern — point its context loader at `retrieval.context()` and keep your LangGraph on top. Our engine replaces / augments `analytics/event_analysis.py`.

Step 4 — Route devices through your gateway into Supabase.

Keep the two ingestion shapes; make them gateway endpoints that write Supabase rows:

- **Daily summaries** → `signals` (one value per day × metric × source).
- **High-frequency vitals** → `vitals_ts` (`{ts, metric, value, source, step_s}`, idempotent upsert).
- WHOOP push and CGM (Nightscout) become webhooks under `app/gateway/webhooks.py`.

Step 5 — Point the mobile app at Supabase.

Our RN app currently calls our FastAPI. For the merged system, its data + auth move to Supabase (Supabase Auth + table reads, same as your Flutter app), while it calls the engine endpoints for analysis (bioage, Sinc, insights). The device native adapters (`mobile/modules/ky-ecg`) keep emitting the same two ingestion shapes.

Step 6 — Prove one feature end-to-end first.

Before the full merge, validate the seam on a single endpoint: wire `bioage.all_ages()` to read a user's labs + signals from Supabase and return `GET /bioage`. If that returns correct PhenoAge/KDM for a real user, the data-layer adapter is sound and the rest follows the same path.

Step 7 — Roll out.

Merge the remaining engines, move the WHOOP/CGM receivers to webhooks, and retire our per-user SQLite + token auth (Supabase RLS replaces it). Run `backend/tools/audit.py` against the merged service to confirm every route + wiring.

3. Device flexibility (important — hardware isn't finalised)

The engine **never depends on a specific device**. It consumes normalised signals and a source-agnostic vitals timeline. **Adding a new/undecided device needs only a small native adapter** that emits either daily summaries or `{ts, metric, value}` samples (and, for sleep, a hypnogram) to the two endpoints above — **zero engine changes**. Multi-device fusion is already built (`mobile/src/lib/sleepfuse.ts`): a configurable source priority, fragment merge, graceful "connect X" when a device is missing. A new ring/patch/watch/CGM just slots into the priority ladder.

4. What we provide vs what we need

We provide	We need from you
Full repo + this plan; the data-layer adapter (Step 2) and a working `/bioage` on your stack as the first proof; the NHANES-validated age models; the RAG + guard + engine.	Supabase reader/dev invite (URL + anon key + service-role key or RLS), the current table schema, and agreement on where the engine lives (`healthagent/app/analytics`).

5. Effort & risk

- **Low risk:** same framework (FastAPI), same AI pattern, single data-layer seam (`db.py`).
- **Main work:** schema mapping (Step 1) — relational tables for labs/signals/vitals vs the current JSONB document.
- **First milestone:** `/bioage` live on your Supabase (proves the seam) — then merge the rest module by module.

Telomy · Architecture handoff · repo: anshumanastronaut/telomy (branch telomy/twin3d-sinc-design-ring) · see docs/ARCHITECTURE_HANDOFF.md, docs/ENGINEERING_DPR.md, docs/research/BIOAGE_RESEARCH.md